As an example of that last benefit, the Shoulda Matchers library defines framework-independent matchers that work with RSpec, Minitest, and Test::Unit.[7]

Use the DSL Unless You're Writing a Library



Most of the time, you'll want to use the DSL to define your matchers. It will save you space, be easier to read, and integrate automatically with other RSpec features.

If you're writing a library of matchers that people will be using to test their own (possibly non-RSpec) projects, you may want to write a custom class instead.

Your Turn

This chapter paved a gradual path into custom matchers. We started off by building custom matchers from RSpec's existing ones, using helper methods, aliases, and negation. Next, we defined a brand-new matcher using RSpec's DSL. Finally, we peeled back the curtain and showed that there's no magic behind matchers. They're just plain Ruby classes.

With a good set of custom matchers, your specs will become more readable. When there's a failure, the improved error message will save you time finding the cause.

Now, it's time to try your hand at writing your own custom matcher.

Exercises

At the top of the chapter, we looked at some hypothetical matchers for a concert ticketing API. In these exercises, you're going to implement these matchers.

To keep things simple, it's fine to put all your code for this exercise in one file. Start out with the `Event` class you'll be testing:

12-creating-custom-matchers/exercises/custom_matcher/spec/event_matchers_spec.rb

```
Event = Struct.new(:name, :capacity) do
  def purchase_ticket_for(guest)
    tickets_sold << guest
  end

  def tickets_sold
    @tickets_sold ||= []
  end
```

7. https://github.com/thoughtbot/shoulda-matchers